

# Custom Suricata Decoders & Rules in Wazuh for Noise Reduction, Precise Detection, and Automated Response

## Project Description

In this project, I implemented **custom JSON decoders and custom correlation rules in Wazuh** to enhance Suricata alert processing, reduce noise, and improve detection accuracy.

Default Suricata + Wazuh integrations often generate **high-volume, low-context alerts**, making SOC analysis difficult. To address this, I created **custom decoders, custom rules, and an enhanced Suricata rule set** that adds MITRE ATT&CK context, improves field extraction, suppresses noisy events, and enables **automated active response** for confirmed malicious behavior.

The detections were validated using a controlled attacker machine (Nmap scripting engine), visualized in Wazuh dashboards, and correlated into **TheHive** for case management and investigation.

## Tools & Technologies Used

- **Wazuh SIEM (Manager & Agent)**
- **Suricata IDS**
- **Custom Wazuh Decoders (XML)**
- **Custom Wazuh Rules & Rule Groups**
- **MITRE ATT&CK Framework**
- **Nmap (Attacker Simulation)**
- **Wazuh Dashboards (OpenSearch)**
- **TheHive (Incident Response & Case Management)**

## Why Custom Decoders & Rules Were Needed

### Problems Observed with Default Setup

- Excessive **noise** from generic Suricata alerts
- Important fields buried inside `full_log`
- Limited MITRE mapping visibility
- Difficulty distinguishing **real attacks vs background traffic**
- Repeated alerts without meaningful correlation

### Goals of This Project

- Extract Suricata JSON fields cleanly
- Add **MITRE Tactic & Technique context**
- Suppress low-value alerts
- Promote high-confidence detections

- Enable **actionable alerts** with active response
- Improve SOC analyst visibility and triage speed

## Step-by-Step Implementation

### Step 1: Creating Custom Suricata Decoders

**Objective:** Properly parse Suricata eve.json logs and extract relevant fields.

#### What Was Done

- Created custom JSON decoders for:
  - src\_ip, src\_port, dest\_ip, dest\_port
  - HTTP fields (User-Agent, URL, method)
  - Alert metadata (signature, severity, category)
- Used parent-child decoder logic to avoid duplication
- Ensured Suricata logs were decoded cleanly instead of relying on raw full\_log

#### 📌 Benefit:

Clean field extraction enables reliable correlation, filtering, dashboards, and active response logic.

```
<decoder name="suricata-json">
  <prematch>^{\s*"</prematch>
</decoder>

<decoder name="suricata-srcip">
  <parent>suricata-json</parent>
  <regex>"src_ip": "([^\"]+)"</regex>
  <order>srcip</order>
</decoder>

<decoder name="suricata-json-child">
  <parent>suricata-json</parent>
  <plugin_decoder>JSON_Decoder</plugin_decoder>
</decoder>
```

### Step 3: Custom Rule Correlation, MITRE ATT&CK Mapping & Noise Reduction

**Objective:** Enrich detections with attack context while reducing alert fatigue.

#### What Was Done

In this step, custom Wazuh rules were enhanced to **correlate decoded Suricata events**, attach **MITRE ATT&CK metadata**, and suppress unnecessary noise.

#### Key Enhancements

- Added **MITRE ATT&CK context** to custom rules:
  - **Tactic:** Reconnaissance

- **Technique:** Active Scanning
  - **Technique ID:** T1595
- Assigned appropriate **rule levels** based on confidence
- Filtered alerts by:
  - Event type
  - Signature relevance
  - Severity
- Suppressed repetitive or low-value Suricata alerts
- Promoted meaningful detections for SOC visibility and response

### Example Outcomes

- Nmap scripting engine detections mapped clearly to Reconnaissance
- Alert descriptions became SOC-friendly and readable
- Reduced false positives and repeated informational alerts

### 📌 Benefit:

Alerts now convey **what is happening, why it matters, and where it fits in the attack lifecycle**, enabling faster analyst decisions.

```
<group name="suricata_custom_active_response_rules">
  <rule id="100200" level="12">
    <if_sid>86600</if_sid>
    <field name="event_type">^alert$</field>
    <match>ET DOS Inbound GoldenEye DoS attack</match>
    <description>GoldenEye DoS attack has been detected. </description>
    <mitre>
      <id>T1498</id>
    </mitre>
  </rule>

  <rule id="100201" level="12">
    <if_sid>86600</if_sid>
    <field name="event_type">^alert$</field>
    <match>ET SCAN Nmap Scripting Engine User-Agent Detected (Nmap Scripting Engine)</match>
    <description>Nmap scripting engine detected. </description>
    <mitre>
      <id>T1595</id>
    </mitre>
  </rule>
</group>
```

## Step 4: Creating a Dedicated Suricata Rules File for Enhanced Detection

**Objective:** Organize and extend Suricata-related detections using a structured rule file.

### What Was Done

A **separate Suricata-specific rules file** was created to improve modularity and scalability of detections.

### Implemented Rule Groups

- suricata\_http
- suricata\_dns
- suricata\_tls
- suricata\_severity

- suricata\_active\_response

## Enhancements Included

- HTTP, DNS, and TLS traffic classification
- Severity-based alert grouping
- Clear separation of:
  - Detection rules
  - Active response triggers
- Better long-term maintainability compared to a single monolithic rule file

## Benefit:

This structure mirrors **real SOC environments**, making rule management cleaner, scalable, and easier to tune.

```
root@ip: ~# cat /var/ossec/ruleset/rules/100002-suricata.xml
<!-- Suricata Rules -->
<group name="suricata,netflow">
  <rule id="100002" level="3">
    <if_sid>86600</if_sid>
    <options>no_full_log</options>
    <description>Suricata Traffic - $(event_type).</description>
  </rule>
</group>
<group name="suricata,suricata_severity2">
  <rule id="100003" level="10">
    <if_sid>86601</if_sid>
    <field name="alert.severity">2</field>
    <options>no_full_log</options>
    <description>Suricata Alert - $(alert.signature). Signature ID: $(alert.signature_id).</description>
  </rule>
</group>
<group name="suricata,suricata_severity1">
  <rule id="100004" level="12">
    <if_sid>86601</if_sid>
    <field name="alert.severity">1</field>
    <options>no_full_log</options>
    <description>Suricata Alert - $(alert.signature). Signature ID: $(alert.signature_id).</description>
  </rule>
</group>
<group name="suricata,suricata_severity3">
  <rule id="100005" level="5">
    <if_sid>86601</if_sid>
    <field name="alert.severity">3</field>
    <options>no_full_log</options>
    <description>Suricata Alert - $(alert.signature). Signature ID: $(alert.signature_id).</description>
  </rule>
</group>
```

```

<group name="suricata,suricata_http">
  <rule id="100006" level="3">
    <if_sid>86602</if_sid>
    <options>no_full_log</options>
    <description>Suricata HTTP Traffic.</description>
  </rule>
</group>
<group name="suricata,suricata_dns">
  <rule id="100007" level="3">
    <if_sid>86603</if_sid>
    <options>no_full_log</options>
    <description>Suricata DNS Traffic.</description>
  </rule>
</group>
<group name="suricata,suricata_tls">
  <rule id="100008" level="3">
    <if_sid>86604</if_sid>
    <options>no_full_log</options>
    <description>Suricata TLS Traffic.</description>
  </rule>
</group>
<group name="suricata,active_response,">
  <rule id="100009" level="3">
    <location>wazuh_suricata</location>
    <description>Wazuh Suricata Blocked IP</description>
    <options>no_full_log</options>
  </rule>
</group>

```

## Step 5: Testing with Attacker Simulation

**Objective:** Validate detections in real conditions.

### What Was Done

- Used **Nmap scripting engine** from attacker machine
- Generated:
  - HTTP-based scan traffic
  - Reconnaissance behavior
- Verified:
  - Decoder field extraction
  - Rule matching
  - MITRE mapping accuracy

### Result:

Custom rules triggered exactly as expected.

```

Starting Nmap 7.98 ( https://nmap.org ) at 2026-01-26 20:28 +0000
Nmap scan report for 10.0.1.208
Host is up (0.00056s latency).
Not shown: 998 closed tcp ports (reset)
PORT      STATE SERVICE
22/tcp    open  ssh
80/tcp    open  http
| http-vuln-cve2011-3192:
|   VULNERABLE:
|   Apache byterange filter DoS
|   State: VULNERABLE
|   IDs: CVE:CVE-2011-3192 BID:49303
|   The Apache web server is vulnerable to a denial of service attack when numerous
|   overlapping byte ranges are requested.
|   Disclosure date: 2011-08-19
|   References:
|   https://www.securityfocus.com/bid/49303
|   https://seclists.org/fulldisclosure/2011/Aug/175
|   https://www.tenable.com/plugins/nessus/55976
|   https://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2011-3192
|_ http-dombased-xss: Couldn't find any DOM based XSS.
|_ http-csrf: Couldn't find any CSRF vulnerabilities.
|_ http-stored-xss: Couldn't find any stored XSS vulnerabilities.
MAC Address: 0A:FF:EB:3F:77:93 (Unknown)

Nmap done: 1 IP address (1 host up) scanned in 74.01 seconds

```

## Step 6: Verification in Wazuh Dashboard & TheHive

### Wazuh Dashboard

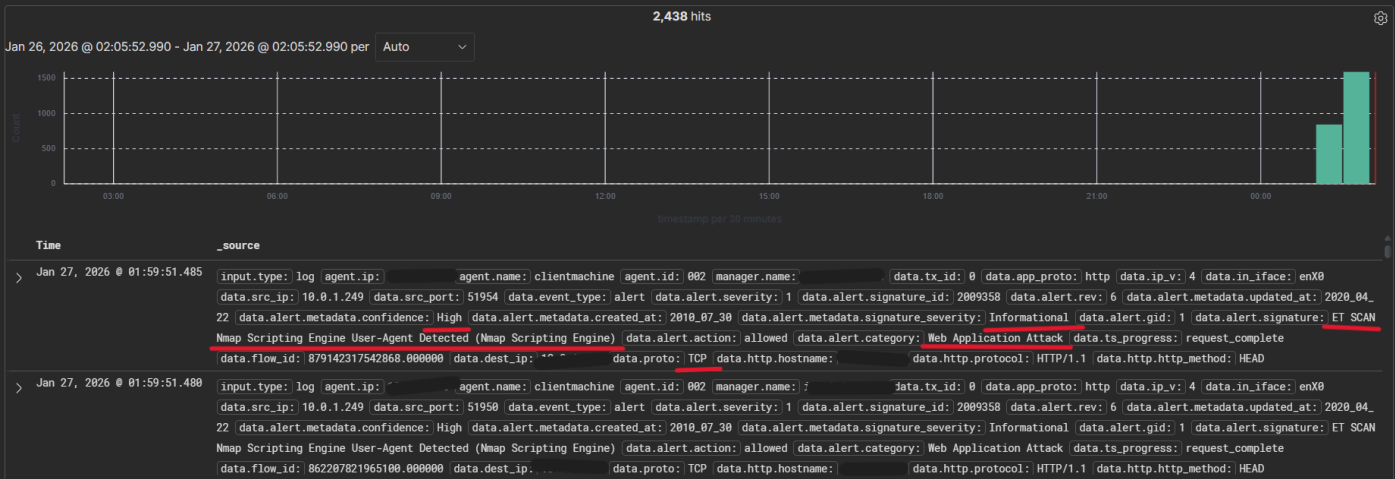
- Alerts displayed with:
  - Signature name
  - Source IP
  - Severity
  - MITRE tactic & technique
- Timeline clearly shows attack burst patterns

### TheHive

- Alerts ingested as cases
- Observables auto-extracted:
  - IPs
  - Domains
  - URLs
- MITRE context visible inside cases
- Ready for escalation or response playbooks

### Benefit:

End-to-end SOC workflow: **Detection** → **Visualization** → **Investigation** → **Response**



|   |                                        |                                                                           |
|---|----------------------------------------|---------------------------------------------------------------------------|
| t | agent.name                             | clientmachine                                                             |
| t | data.alert.action                      | allowed                                                                   |
| t | data.alert.category                    | Web Application Attack                                                    |
| t | data.alert.gid                         | 1                                                                         |
| t | data.alert.metadata.confidence         | High                                                                      |
| t | data.alert.metadata.created_at         | 2010_07_30                                                                |
| t | data.alert.metadata.signature_severity | Informational                                                             |
| t | data.alert.metadata.updated_at         | 2020_04_22                                                                |
| t | data.alert.rev                         | 6                                                                         |
| t | data.alert.severity                    | 1                                                                         |
| t | data.alert.signature                   | ET SCAN Nmap Scripting Engine User-Agent Detected (Nmap Scripting Engine) |
| t | data.alert.signature_id                | 2009358                                                                   |
| t | data.app_proto                         | http                                                                      |
| t | data.dest_ip                           | 10.0.1.249                                                                |
| t | data.dest_port                         | 80                                                                        |

|   |                             |                                                                                 |
|---|-----------------------------|---------------------------------------------------------------------------------|
| t | data.http.http_content_type | text/html                                                                       |
| t | data.http.http_method       | HEAD                                                                            |
| t | data.http.http_user_agent   | Mozilla/5.0 (compatible; Nmap Scripting Engine; https://nmap.org/book/nse.html) |
| t | data.http.length            | 0                                                                               |
| t | data.http.protocol          | HTTP/1.1                                                                        |
| t | data.http.status            | 404                                                                             |
| t | data.http.url               | /sitecore%20modules/staging/workdir                                             |
| t | data.in_iface               | enX0                                                                            |
| t | data.ip_v                   | 4                                                                               |
| t | data.pkt_src                | wire/pcap                                                                       |
| t | data.proto                  | TCP                                                                             |
| t | data.src_ip                 | 10.0.1.249                                                                      |
| t | data.src_port               | 51950                                                                           |
| t | data.tc_progress            | response_complete                                                               |

|   |                      |                                       |
|---|----------------------|---------------------------------------|
| t | location             | /var/log/suricata/eve.json            |
| t | manager.name         | ip-10-0-1-100                         |
| t | rule.description     | Nmap scripting engine detected.       |
| # | rule.firedtimes      | 1,593                                 |
| t | rule.groups          | suricata_custom_active_response_rules |
| t | rule.id              | 100201                                |
| # | rule.level           | 12                                    |
| 🕒 | rule.mail            | true                                  |
| t | rule.mitre.id        | T1595                                 |
| t | rule.mitre.tactic    | Reconnaissance                        |
| t | rule.mitre.technique | Active Scanning                       |

Alerts

Enter a case number

+ Create Case

<



## Project Outcome

- Successfully reduced Suricata alert noise
- Improved alert context and readability
- Enabled MITRE-aligned detection logic
- Validated detections with real attack traffic
- Integrated alerts into TheHive for IR workflows
- Built production-style SOC tuning experience

## Skills Gained

- Wazuh decoder engineering
- Advanced rule writing & correlation
- Suricata IDS tuning
- MITRE ATT&CK mapping
- SOC noise reduction techniques
- Threat detection validation
- SIEM → IR platform integration

## Conclusion

This project demonstrates **real-world SOC engineering skills**, not just tool setup. By designing custom decoders and rules, I transformed noisy IDS output into **actionable, contextual security intelligence**, aligned with MITRE ATT&CK and integrated into incident response workflows.

This is the same process used by SOC teams to harden detections in production environments.